MiniOS Ekoms

Technical Documentation

Operating Systems Final Project

Group 8 - SC1063

Group Members

1. Necesario - Project Leader and CLI Developer
2. Panado - File Management Developer
3. Pagdato - Process and Memory Developer
4. Ciupan - User/System Developer

1. Introduction

MiniOS Ekoms is a command-line operating system simulator written in Python for the Operating Systems final project. It runs in Windows CMD and demonstrates the main parts of an operating system: shell, files, processes, memory, users, and system information. It is a simulation; it does not use a real kernel, drivers, or hardware.

2. Project Objectives

• Build a working command-line shell with its own prompt and commands.

• Simulate a hierarchical file system with directories and files.

• Simulate processes using a process table and PIDs.

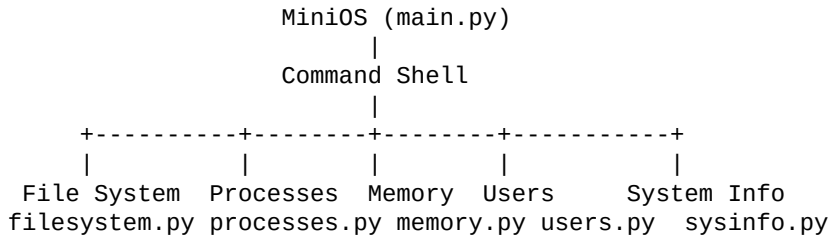
• Simulate memory allocation and release with limit checking.

• Provide a login system with at least two user accounts.

• Handle errors without crashing, and document and test the system.

3. System Architecture

The **shell** in `main.py` reads commands and passes them to the correct module. Each module is a separate Python class with one responsibility.



Component	File	Responsibility
Command Shell	main.py	Prompt, command parsing, command table, error handling
File System	filesystem.py	Virtual directories and files, saved to minios_disk.json
Processes	processes.py	Process table, PIDs, run and kill
Memory	memory.py	Total/used/free memory, allocate and free
Users	users.py	Accounts, hashed passwords, login
System Info	sysinfo.py	Simulated system information text
Tests	tests/test_minios.py	Automated unit tests

4. Program Structure

When MiniOS starts, `main.py` creates one `MiniOS` object. It builds a FileSystem, ProcessManager, MemoryManager, and UserManager, and a dictionary called `commands` that maps each command name to a method. After login, the shell loop repeats: read a line, split it into

the command and its arguments, look the command up in the dictionary, and run it. If the name is not in the dictionary, it prints *Unknown command*.

```
MiniOS/
  main.py  users.py  filesystem.py  processes.py  memory.py  sysinfo.py
  README.md  requirements.txt
  tests/test_minios.py
  docs/  screenshots/  individual-reports/
```

5. File Management

The file system is a tree of Python dictionaries. A directory is `{"type": "dir", "children": {...}}` and a file is `{"type": "file", "content": "..."}`. The current directory is kept as a list of names. Paths such as `..` and `/` are resolved by `_normalize()`, and `_node()` walks the tree to find the target. After every change the tree is saved to `minios_disk.json`, so files persist between runs. Real Windows files are never touched. Commands: dir, mkdir, create, read, write, delete, rename, cd, pwd. Errors such as a missing file, a duplicate name, or deleting a non-empty folder raise a FileSystemError that the shell displays as a message.

6. Process Management

Each process is a small Process Control Block (PCB) object holding a PID, a name, a status (RUNNING), and a start time. Processes are stored in a dictionary called the process table, keyed by PID. `run` creates a PCB with the next PID, `process` lists the table, and `kill` removes a PID. The table has a limit of 20 processes. Killing an unknown PID or a non-numeric PID raises a ProcessError. No real Windows process is started.

7. Memory Management

Memory is simulated with counters. Total memory is 1024 MB, and 256 MB is always reserved for the system. `allocate` checks that the amount is a positive whole number and does not exceed the free memory, then adds it to the allocated total. `free` checks that the amount is valid and not larger than what was allocated. `memory` shows total, used, and free memory with a usage bar. Real RAM is never used.

8. User Management

Two accounts exist: admin (Administrator) and student (Standard User). Passwords are never stored as plain text; they are stored as SHA-256 hashes, and at login the typed password is hashed and compared. On Windows the password is shown as asterisks. The user has 3 attempts before the system locks. `whoami` shows the current user, `users` lists accounts, and `logout` returns to the login screen.

9. Error Handling

Each command runs inside a try/except block in the shell. Unknown commands print *Unknown command* with a hint to type help. Module errors (FileSystemError, ProcessError, MemoryError_) are caught and shown as *Error: ...*. Missing arguments show the correct usage. A last-resort catch

for unexpected errors keeps MiniOS from crashing. Pressing Ctrl+C or closing input shuts down MiniOS cleanly.

10. Testing

Automated unit tests are in `tests/test_minios.py` and run with:

```
python -m unittest discover tests
```

They cover creating, reading, and deleting files; making and entering directories; duplicate names; running and killing processes; invalid PIDs; allocating and freeing memory; invalid and oversized memory amounts; and valid and invalid logins. Manual testing was done by running every command in CMD, including invalid input.

[GROUP TO COMPLETE - write in your own words] Add your own testing notes here: who tested what, what you tried, and what you found. Insert your screenshots (startup, login, help, files, processes, memory, systeminfo, error, exit) or refer to the screenshots folder.

11. Problems Encountered

[GROUP TO COMPLETE - write in your own words] List the real problems your group had (for example Git/GitHub uploads, Python setup, merging files, teamwork) and how you solved them. This must be your own experience.

12. Conclusion

[GROUP TO COMPLETE - write in your own words] Explain what your group learned about shells, file systems, processes, memory, and users, and what you would improve.

Individual Module Notes

Each member should add a short paragraph about their own module (what it does, how it works, one problem they solved).

Member	Role / Module	Notes
Necesario	Project Leader and CLI (main.py, README, project merge)	
Panado	File Management (filesystem.py)	
Pagdato	Process and Memory (processes.py, memory.py)	
Ciupan	User/System, Error Handling, Testing (users.py, sysinfo.py, tests)	